



Вселенная

Программирование на Python и SQL 7 в 1

Автор: Фарелл Херст

Тип файла: pdf

Размер: 952 КБ

Язык: Английский

Страниц: 152

Программирование на Python и SQL 7 в 1: полное руководство для студентов и специалистов

Введение

Современная инженерия все больше опирается на данные, автоматизацию, моделирование и интеллектуальное программное обеспечение. Независимо от того, работаете ли вы в сфере машиностроения, гражданского строительства, электротехники, производства, энергетики, строительства или инженерного менеджмента, умение **программировать, обрабатывать данные и взаимодействовать с базами данных** может значительно повысить вашу продуктивность.

Python и SQL — особенно мощная комбинация.

Python — это язык программирования общего назначения, который используется для автоматизации, численных расчетов, анализа данных, научных вычислений, искусственного интеллекта, машинного обучения и инженерных приложений. **SQL (язык структурированных запросов)** предназначен для работы со структурированными данными, хранящимися в реляционных базах данных.

Представьте себе такую взаимосвязь:

Python = система инженерного управления 

SQL = система организованного хранения данных 

Python может рассчитывать напряжения, обрабатывать показания датчиков, анализировать лабораторные измерения, автоматизировать составление отчетов или создавать прогностические модели. SQL может быстро и систематически извлекать тысячи или миллионы технических записей из баз данных.

Подход к обучению **программированию на Python и SQL 7-в-1** объединяет эти навыки в один практический курс, а не рассматривает программирование и базы данных как совершенно разные предметы.





Data Cleaning

```
import pandas as pd
df = pd.read_csv('data.csv')
df.dropna(inplace=True)
df['price'] = df['price'].astype(float)
```

Data Visualization

```
import matplotlib.pyplot as plt
plt.plot(x, y, marker='o')
plt.title('Sales Trend')
plt.show()
```

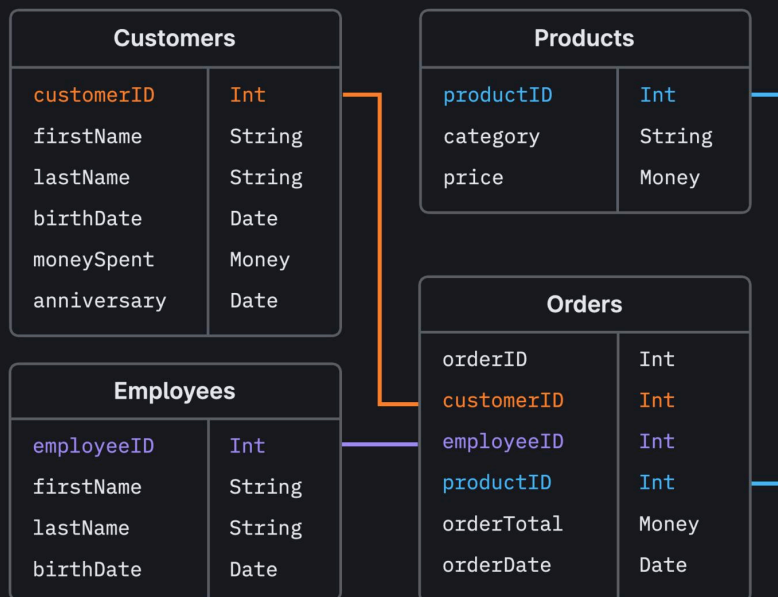
Automation Script

```
def send_emails():
    for email in email_list:
        send_email(email)
```

Mini Project

```
def analyze_data(df):
    result = df['value'].mean()
    print('Average Value:', result)
    return result
```

soniya murugesan



Working with Tables

product
id
product_name
current_price
description
if_available

Customer
id
first_name
last_name
city_id
delivery_address
email
phone
confirmation_code

Customer
id
Product_id
total_price
city_id
payment_method_id
confirmation_code

product
city_id
delivery_address
delivery_product
expected_time_delivery
current_delivery_status

Теоретическая база ⚙️

Для решения инженерных задач традиционно использовались уравнения, электронные таблицы, калькуляторы, лабораторные измерения и техническое программное обеспечение. Сегодня инженерные процессы все чаще включают в себя работу с большими массивами данных и автоматизированные процессы.

Например, представьте себе производственную компанию, которая собирает:

- 🌡️ Измерение температуры
- ⚙️ Вибрация станка
- ⚡️ Электрический ток
- 📈 Производительность
- ⌚ Время работы
- 🛠️ Записи о техническом обслуживании
- ⚠️ Неисправности

Данные могут храниться в реляционной базе данных.

SQL может извлекать необходимые записи:

```
1 | SELECT machine_id, temperature, vibration
2 | FROM sensor_data
3 | WHERE temperature > 80;
```

Затем Python может проанализировать полученную информацию:

```
1 | for row in records:  
2 |     print(row)
```

Настоящая мощь проявляется, когда эти две технологии работают вместе.

Почему Python важен в инженерии

Python обладает относительно понятным синтаксисом и обширной экосистемой научных и инженерных библиотек. Его можно использовать для:

- Numerical calculations
- Data processing
- Automation
- Statistical analysis
- Machine learning
- Simulation
- Visualization
- File processing
- API development
- Engineering optimization

Development environments such as VS Code also support Python, notebooks, data analysis, and visualization workflows. (code.visualstudio.com)

Why SQL Matters

SQL specializes in retrieving and manipulating structured information.

Instead of manually searching thousands of rows, an engineer can ask the database a precise question:

```
1 | SELECT AVG(temperature)  
2 | FROM sensor_data  
3 | WHERE machine_id = 15;
```

The database performs the operation and returns the result.

Definition

Python Programming and SQL 7-in-1 can be understood as an integrated learning framework covering seven connected technical skills:

Module	Main Skill	Engineering Value
1	Python Fundamentals	Build programs and automate tasks
2	Python Data Structures	Organize engineering information
3	Functions & OOP	Build reusable engineering software
4	Data Analysis	Understand measurements and datasets
5	SQL Fundamentals	Retrieve database information
6	Advanced SQL	Analyse complex relational data
7	Python + SQL Integration	Build complete data-driven applications

The objective is not simply to memorize Python syntax or SQL commands.

The objective is to understand **how computation, data, and databases work together**.

Step-by-Step Learning Path

Step 1: Learn Python Fundamentals

Start with variables, data types, operators, conditions, loops, and basic input/output.

```

1 | temperature = 72.5
2 | limit = 70
3 |
4 | if temperature > limit:
5 |     print("Warning: Temperature exceeded limit")
6 | else:
7 |     print("System operating normally")

```

This simple example already represents an engineering control decision.

You should become comfortable with:

- int
- float

- str
- bool
- if
- for
- while
- Lists
- Tuples
- Dictionaries
- Sets

Step 2: Master Functions

Functions allow engineers to convert repeated calculations into reusable components.

```

1 | def calculate_stress(force, area):
2 |     return force / area
3 |
4 | stress = calculate_stress(5000, 250)
5 | print(stress)

```

The relationship is:

```

[
\sigma = \frac{F}{A}
]

```

where:

- (σ) = stress
- (F) = applied force
- (A) = cross-sectional area

Instead of rewriting the equation repeatedly, the function can be reused throughout a project.

Step 3: Learn Data Structures

Engineering applications rarely process one value. They usually work with collections of measurements.

```

1 | temperatures = [68.2, 70.5, 72.1, 71.8, 69.7]
2 |
3 | average = sum(temperatures) / len(temperatures)
4 |

```

```
5 | print(average)
```

Dictionaries are particularly useful for structured engineering records:

```
1 | machine = {  
2 |     "id": 101,  
3 |     "temperature": 75.4,  
4 |     "pressure": 2.8,  
5 |     "status": "Active"  
6 | }
```

Step 4: Move Into Data Analysis

Once Python fundamentals are comfortable, begin working with datasets.

A typical workflow is:

Raw Data → Cleaning → Transformation → Analysis → Visualization → Decision



Import the CSV File

(Data Ingestion)



Clean the Data

(Handle Missing Values,
Duplicates, etc.)



Data Transformation

(Engineer Features,
Aggregate)

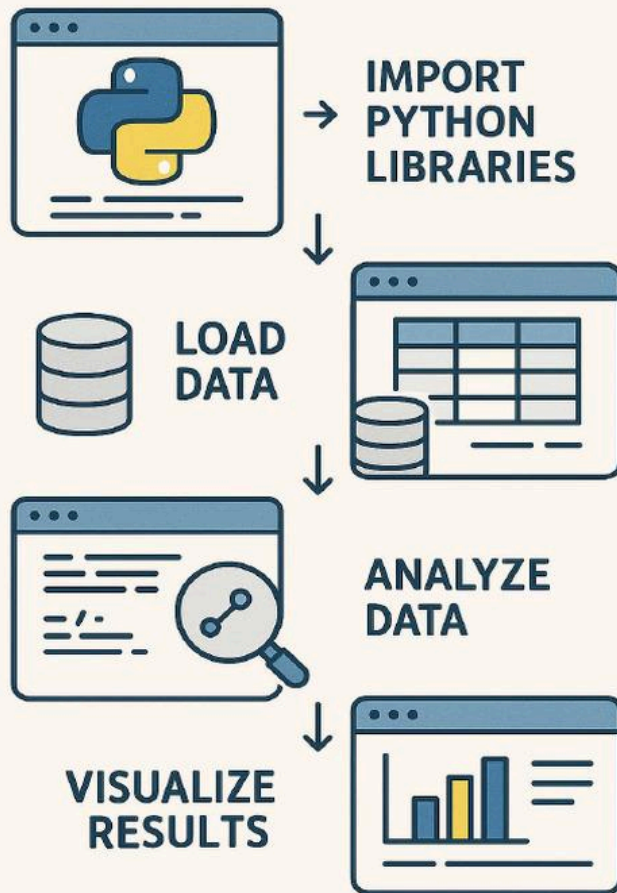


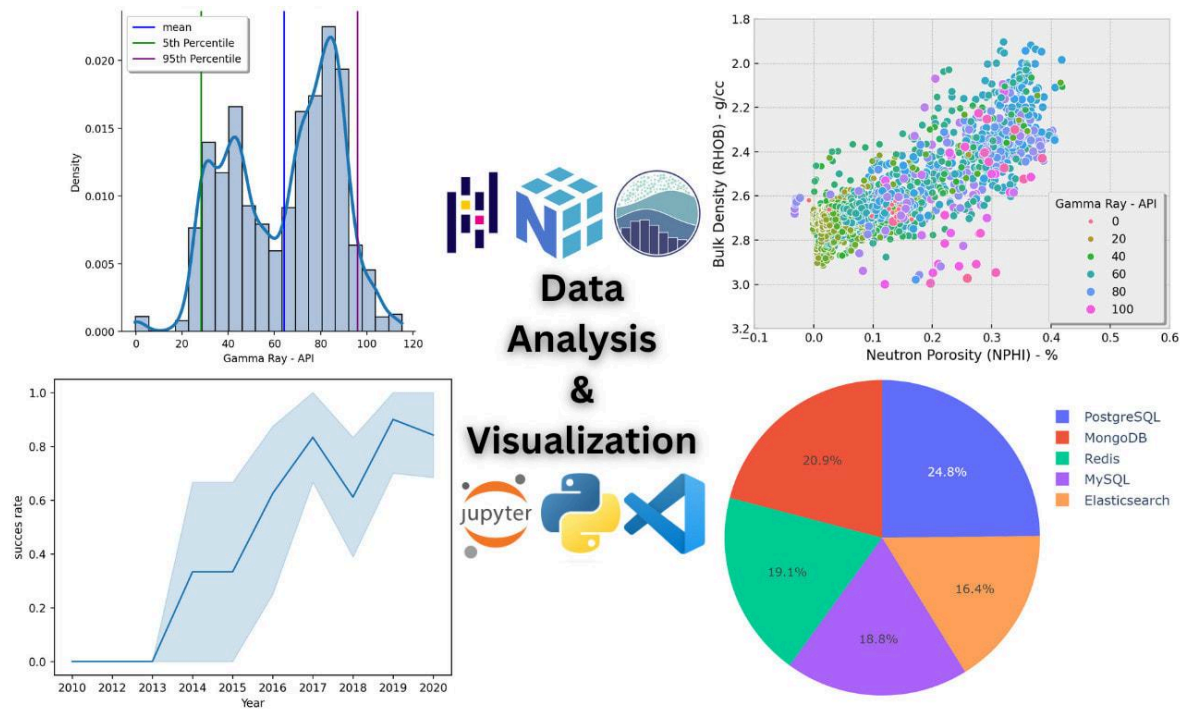
Create a Dashboard

(Visualize Results)

Building a Data Pipeldag in Python

HOW DATA ANALYSTS USE PYTHON





```
import pandas as pd
import matplotlib.pyplot as plt

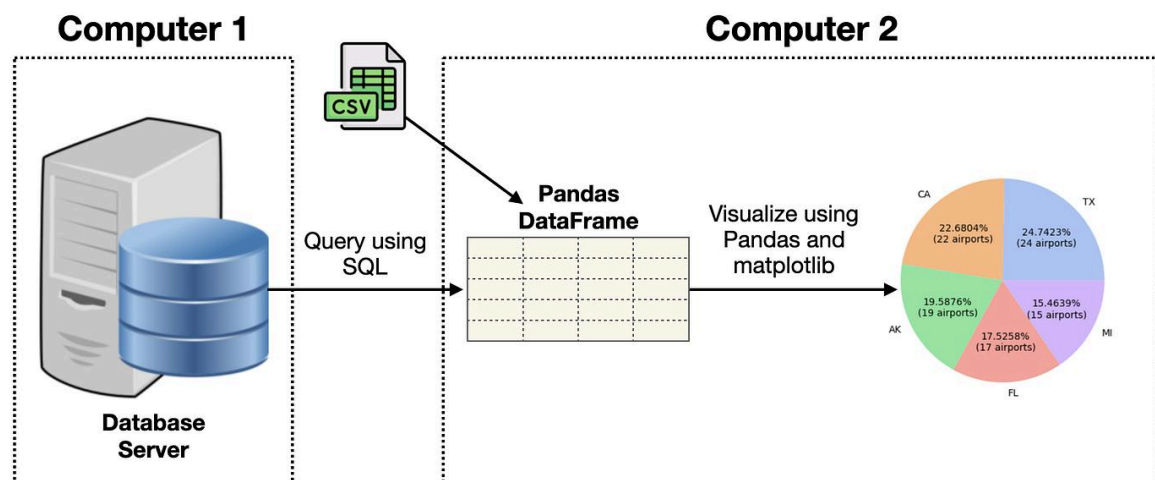
# Load data into pandas dataframe
df = pd.read_csv("construction_data.csv")

# Clean data by dropping any rows with missing values
df = df.dropna()

# Plot a histogram of project sizes
plt.hist(df['project_size'], bins=10)
plt.xlabel("Project Size (in thousands of dollars)")
plt.ylabel("Frequency")
plt.title("Histogram of Project Sizes")
plt.show()

# Calculate the average project size by location
avg_size_by_location = df.groupby('location').mean()['project_size']

# Plot a bar chart of average project sizes by location
plt.bar(avg_size_by_location.index, avg_size_by_location.values)
plt.xlabel("Location")
plt.ylabel("Average Project Size (in thousands of dollars)")
plt.title("Average Project Size by Location")
plt.show()
```



For example, Pandas can load tabular data:

```
1 | import pandas as pd
2 |
3 | df = pd.read_csv("machine_data.csv")
4 |
5 | print(df.head())
6 | print(df.describe())
```

This allows engineers to investigate distributions, missing values, averages, minimums, maximums, and relationships between variables.

Step 5: Learn SQL Fundamentals

SQL starts with a small number of highly important concepts.

The most important command is SELECT.

```
1 | SELECT *
2 | FROM machines;
```

You can select specific columns:

```
1 | SELECT machine_id, model, operating_hours
2 | FROM machines;
```

Then filter records:

```
1 | SELECT *
2 | FROM machines
3 | WHERE operating_hours > 5000;
```

Step 6: Learn Relationships and JOINS

Real engineering databases rarely contain everything inside one table.

For example:

Machines

machine_id	model
101	M-100
102	M-200

Maintenance

maintenance_id	machine_id	cost
1	101	450
2	102	720

The `machine_id` connects the tables.

```

1 | SELECT m.model, mt.cost
2 | FROM machines m
3 | JOIN maintenance mt
4 | ON m.machine_id = mt.machine_id;
```

This concept is fundamental to relational database engineering.

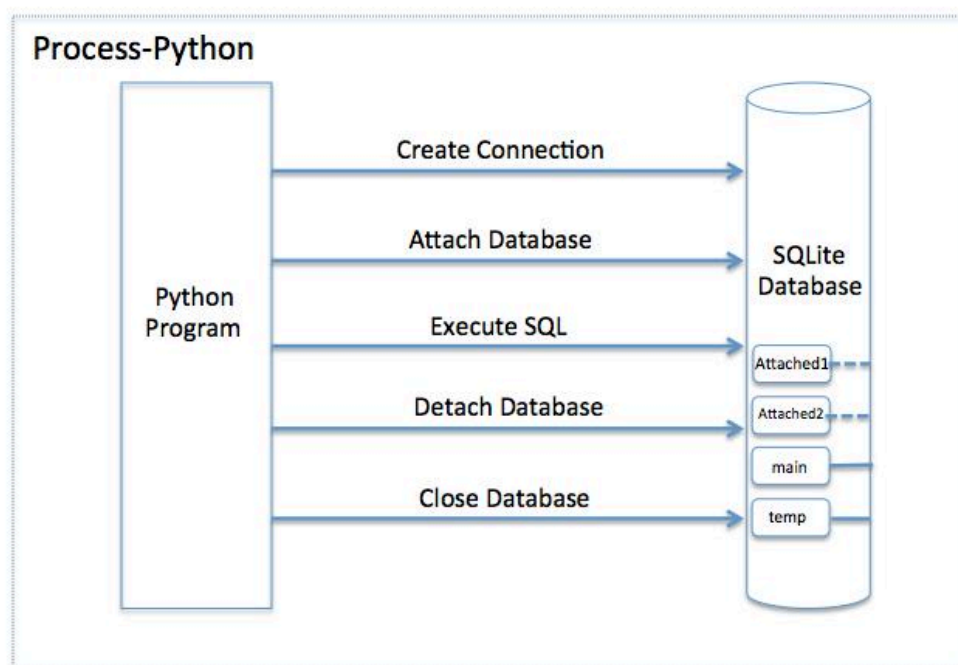
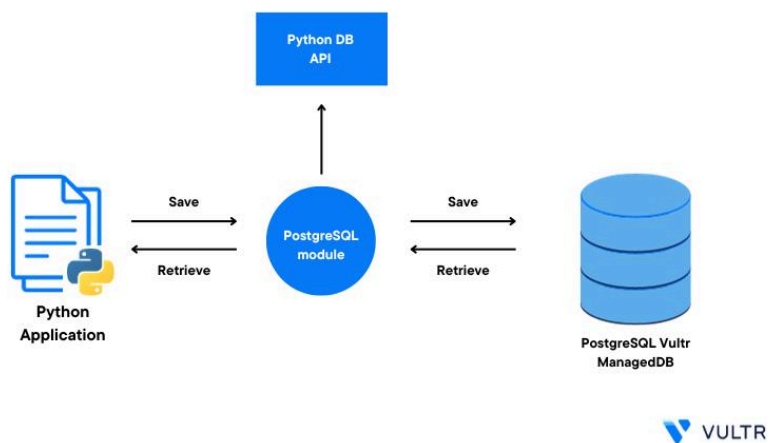
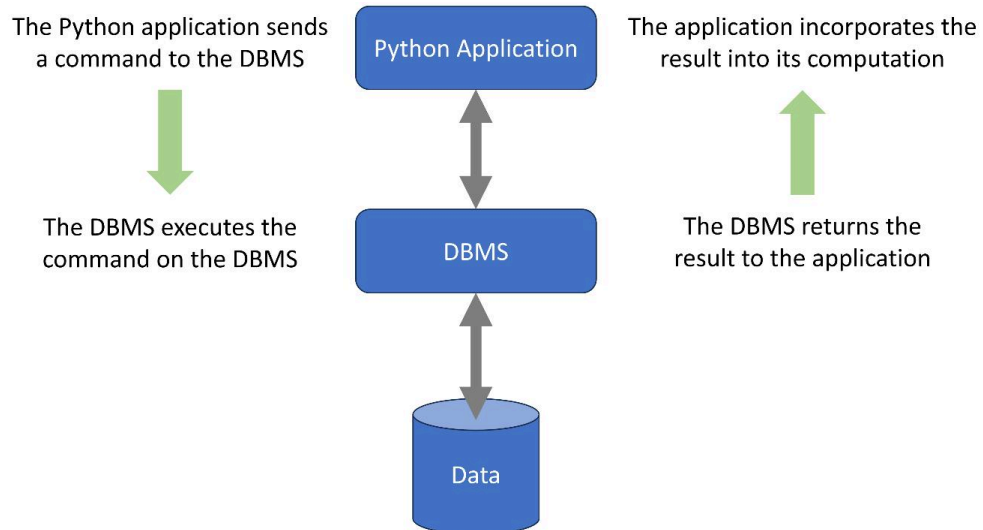
Step 7: Connect Python to SQL

The final stage is connecting your application to the database.

The general workflow is:

```

1 | Python Application
2 |     ↓
3 | Database Connector
4 |     ↓
5 | SQL Query
6 |     ↓
7 | Database Management System
8 |     ↓
9 | Tables / Records
10 |    ↓
11 | Results
12 |    ↓
13 | Python Analysis
```



For example:

```
1 | import sqlite3
2 |
3 | connection = sqlite3.connect("engineering.db")
4 |
5 | cursor = connection.cursor()
6 |
7 | cursor.execute("""
8 | SELECT machine_id, temperature
9 | FROM sensors
10 | WHERE temperature > 80
11 | """)
12 |
13 | results = cursor.fetchall()
14 |
15 | for row in results:
16 |     print(row)
17 |
18 | connection.close()
```

The important concept is the workflow rather than memorizing a particular connector.

Comparison: Python vs SQL

Python and SQL are complementary rather than direct competitors.

Feature	Python 	SQL 
Primary purpose	Programming	Database querying
Calculations	Excellent	Good for database calculations
Automation	Excellent	Limited
Data retrieval	Good with libraries	Excellent
Machine learning	Excellent	Limited
Database management	Through libraries/tools	Core capability

Feature	Python 🐍	SQL 🗄️
Visualization	Excellent	Limited
Complex algorithms	Excellent	Not its primary purpose
Large relational datasets	Good with database tools	Excellent
Engineering applications	Very broad	Mainly data-related

When Should Engineers Use Python?

Use Python when the problem requires:

- Algorithms
- Automation
- Simulation
- Numerical analysis
- Machine learning
- Data visualization
- Custom engineering applications

When Should Engineers Use SQL?

Use SQL when you need to:

- Retrieve database records
- Filter large datasets
- Combine tables
- Aggregate information
- Update database records
- Build database reports

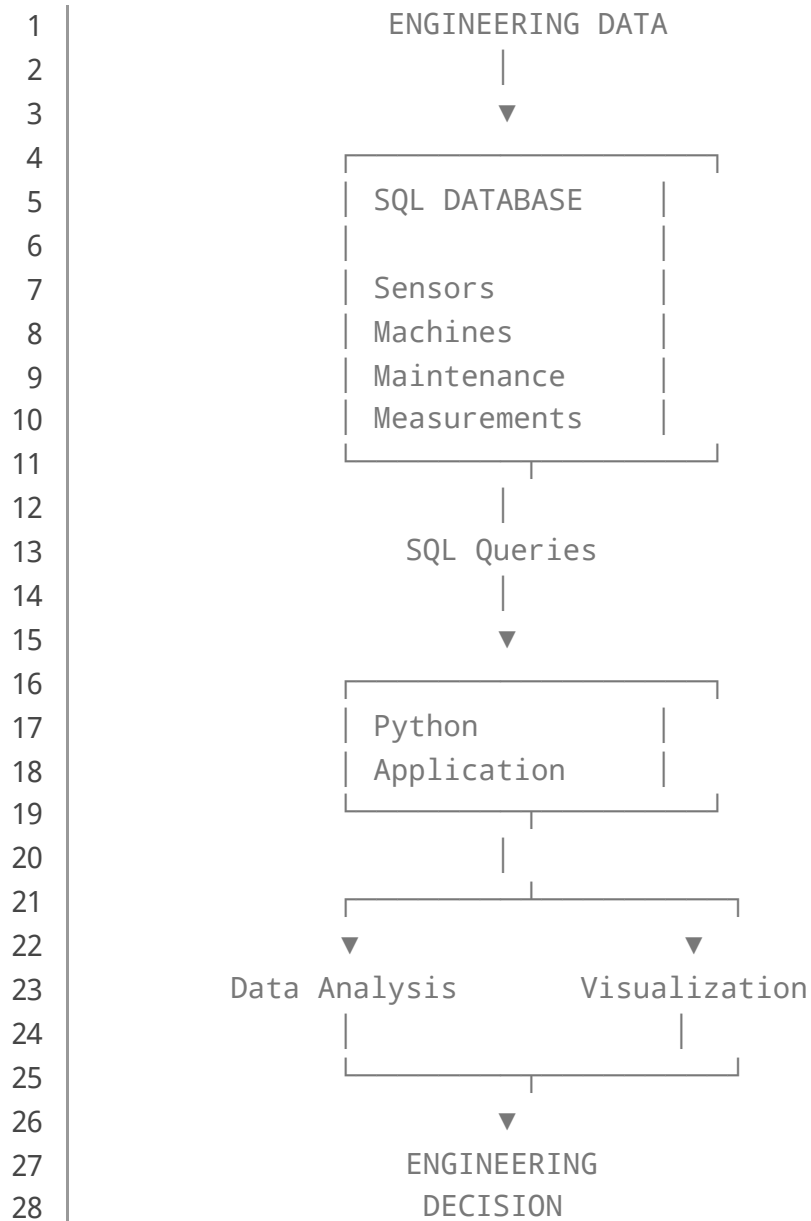
When Should You Use Both?

When an engineering workflow involves **large stored datasets plus computational analysis**.

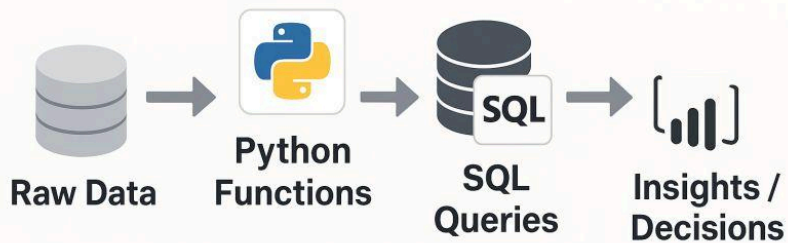
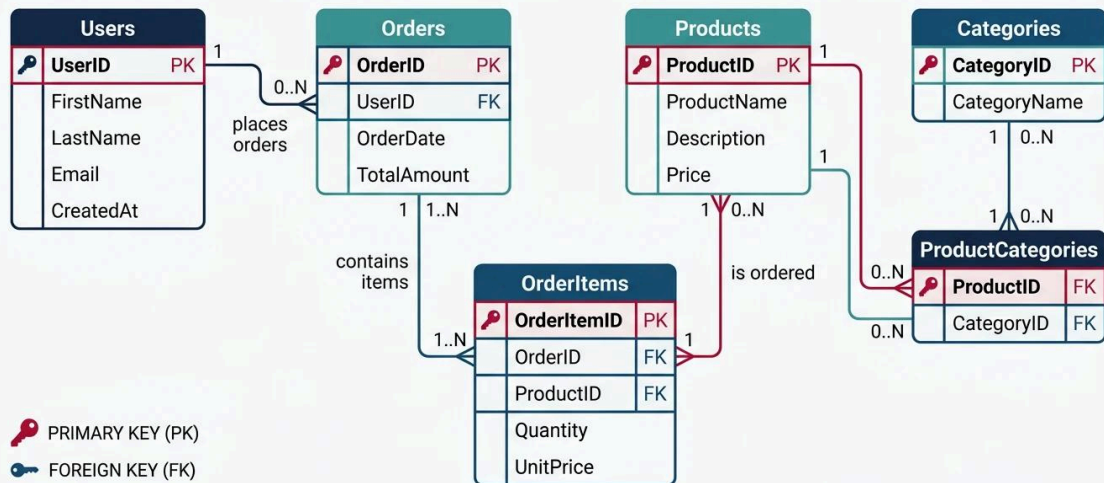
That is where Python + SQL becomes particularly valuable. 🔥

Diagrams, Tables, and Data Flow

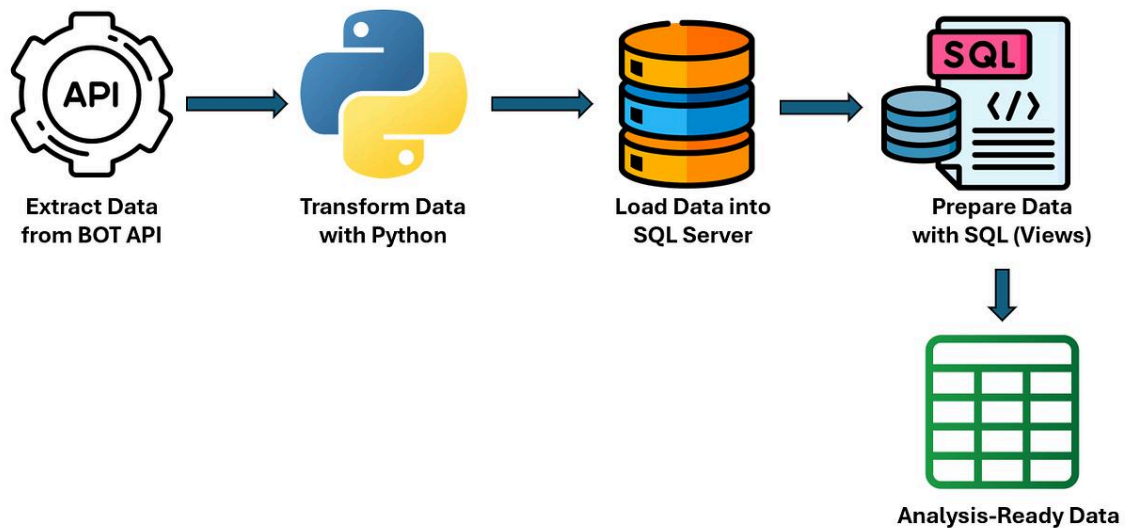
A practical Python-SQL architecture can be represented as:



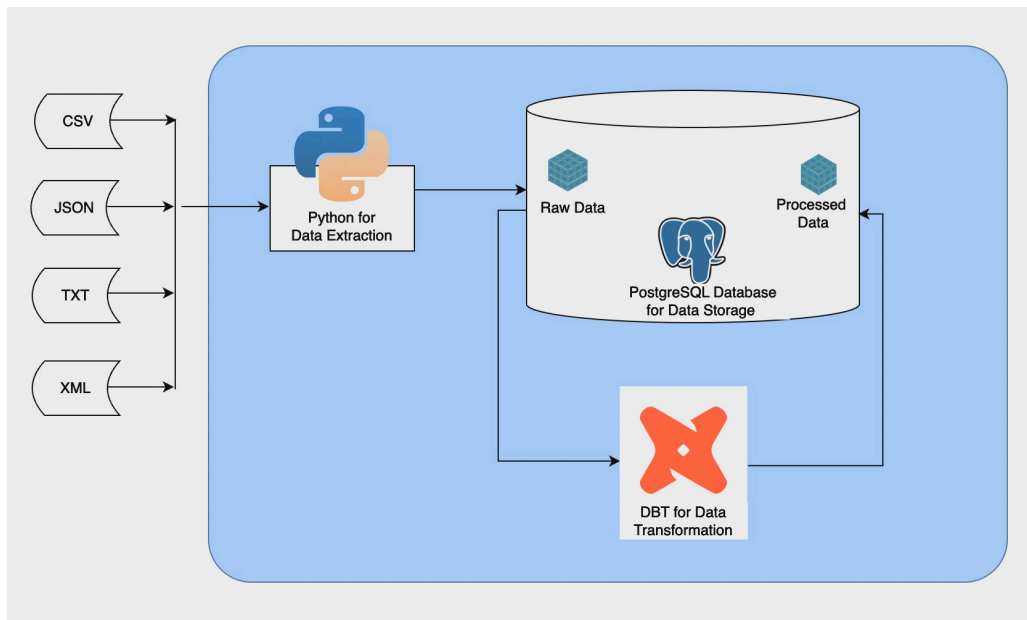
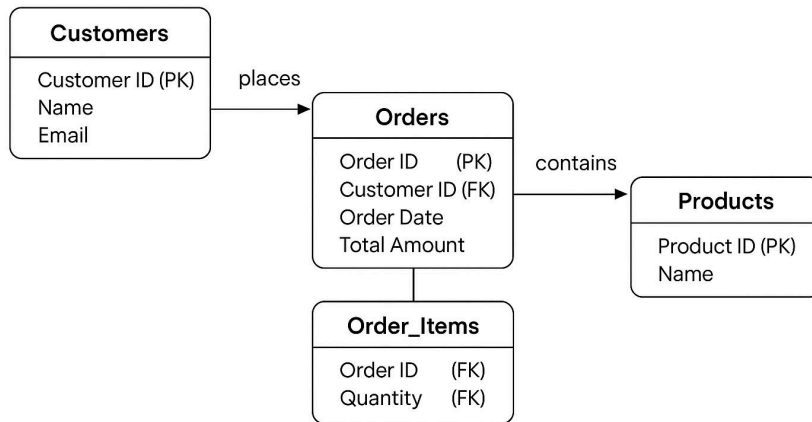
RELATIONAL DATABASE SCHEMA DIAGRAM



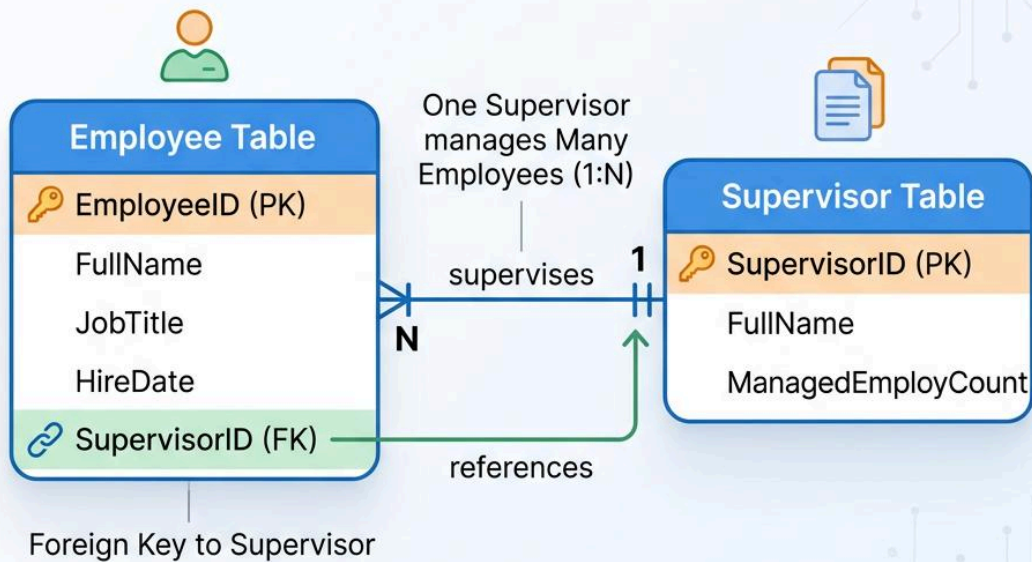
From code to clarity:
integration creates impact.



ER Diagram Example



Entity Relationship ER Diagram



🔑 Primary Key 🔗 Foreign Key —|— Cardinality —> Foreign Key

Example Database Structure

Table	Important Fields	Purpose
machines	ID, model, location	Machine information
sensors	ID, machine_id, temperature	Sensor readings
maintenance	ID, machine_id, date, cost	Maintenance history
engineers	ID, name, department	Personnel
projects	ID, project_name, status	Project tracking

A relational structure prevents engineers from storing everything in one enormous table.

Examples

Example 1: Engineering Stress Calculator

```
1 | force = 12000
2 | area = 400
3 |
4 | stress = force / area
5 |
6 | print(f"Stress = {stress} MPa")
```

This produces:

```
[
\Sigma = 30\MPa
]
```

Example 2: SQL Equipment Search

```
1 | SELECT machine_id, model
2 | FROM machines
3 | WHERE status = 'Active';
```

This retrieves only operational machines.

Example 3: Average Temperature

```
1 | SELECT AVG(temperature) AS average_temperature
2 | FROM sensors;
```

The database calculates the mean directly.

Example 4: Python-SQL Analysis

Imagine a database contains 10 million sensor measurements.

Python does not necessarily need to retrieve all 10 million rows.

Instead, SQL can first reduce the dataset:

```
1 | SELECT machine_id, AVG(temperature) AS avg_temp
2 | FROM sensors
~ |
```

```
3 | GROUP BY machine_id;
```

Python can then analyse the much smaller result.

This is an important professional principle:

Let the database do database work, and let Python do computational work.

Real-World Applications

Mechanical Engineering

Python and SQL can support:

- Predictive maintenance
- Vibration analysis
- Equipment monitoring
- Failure prediction
- Production optimization

A database can store machine measurements while Python identifies abnormal operating patterns.

Civil Engineering

Applications include:

- Structural monitoring
- Construction project databases
- Material testing
- Survey data processing
- Cost analysis
- Infrastructure monitoring

For example, thousands of sensor readings from a bridge can be stored in SQL and analysed with Python.

Electrical Engineering

Possible applications include:

- Power consumption analysis
- Fault detection
- Smart-grid data
- Equipment monitoring
- Load forecasting

Manufacturing

A production system might combine:

```
[  
  \text{Sensors} \rightarrow \text{SQL} \rightarrow \text{Python} \rightarrow  
  \text{Prediction}  
]
```

This can support quality control and predictive maintenance.

Energy Engineering

Python can analyse:

- Energy consumption
- Solar generation
- Wind data
- Battery performance
- Demand profiles

SQL can provide historical records needed for long-term analysis.

Common Mistakes

Mistake 1: Learning Syntax Without Projects

Memorizing:

```
1 | for x in range(10):
```

does not automatically create programming ability.

Build engineering projects instead.

Mistake 2: Ignoring SQL

Some Python learners focus entirely on Python and avoid databases.

That becomes a problem when applications need persistent, structured data.

Mistake 3: Using Python for Everything

You could retrieve millions of database rows and filter them in Python.

But often it is better to let SQL perform the filtering first.

Mistake 4: Poor Database Design

Putting unrelated information into one massive table creates duplication and maintenance problems.

Mistake 5: Ignoring Data Validation

Engineering decisions require trustworthy data.

Always check:

- Missing values
- Impossible measurements
- Duplicate records
- Incorrect units
- Outliers
- Timestamp problems

Challenges & Solutions

Challenge	Solution
Python syntax feels difficult	Build small programs daily
SQL JOINS are confusing	Draw relationships between tables
Large datasets are slow	Filter and aggregate with SQL
Database structure is unclear	Create ER diagrams

Challenge	Solution
Code becomes repetitive	Use functions and classes
Data contains errors	Build validation and cleaning steps
Application becomes difficult to maintain	Separate database, logic, and interface layers
Queries become slow	Study indexing and query optimization

Performance Challenge

Consider:

```
1 | SELECT *
2 | FROM sensor_data;
```

If the table contains millions of records, this may transfer much more information than necessary.

Instead:

```
1 | SELECT machine_id, temperature
2 | FROM sensor_data
3 | WHERE temperature > 80;
```

The second approach can significantly reduce unnecessary data movement.

Case Study: Predictive Maintenance System

Consider an industrial facility with 500 machines.

Each machine generates:

- Temperature
- Vibration
- Pressure
- Operating hours

- Energy consumption

The system stores these records in an SQL database.

Stage 1 — Data Collection

Sensors continuously produce measurements.

Stage 2 — Database Storage

SQL tables organize the measurements by machine and timestamp.

Stage 3 — Data Retrieval

Python sends a query requesting recent readings.

Stage 4 — Data Processing

Python cleans the measurements and calculates statistical indicators.

For example:

```
[  
\mu = \frac{1}{n} \sum_{i=1}^n x_i  
]
```

where μ represents the mean measurement.

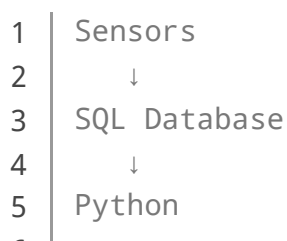
Stage 5 — Anomaly Detection

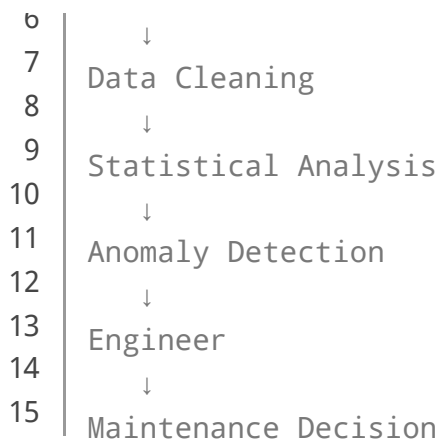
Python can identify machines operating outside expected ranges.

Stage 6 — Engineering Decision

If vibration and temperature increase simultaneously, the system could flag the machine for inspection.

The complete workflow becomes:





This illustrates why learning Python and SQL together is much more valuable than treating them as isolated technologies.

Essential Tips for Learning Python + SQL



Build Engineering Projects

Instead of creating only generic applications, try:

- Beam stress calculators
- Temperature monitoring systems
- Energy dashboards
- Equipment maintenance databases
- Engineering unit converters
- Sensor-data analysis tools
- Project cost databases

Learn in Layers

A strong progression is:

**Python Fundamentals → Data Structures → Functions → OOP → Data Analysis → SQL
→ Advanced SQL → Python-SQL Projects**

Practice SQL Every Week

Focus on:

```
1 | SELECT  
2 | WHERE  
3 |
```

4	ORDER BY
5	GROUP BY
6	HAVING
7	JOIN
8	INSERT
9	UPDATE
	DELETE

Then progress to:

- Subqueries
- Common table expressions
- Window functions
- Views
- Indexes
- Transactions
- Query optimization

Understand the Data Model

Do not only ask:

“What SQL command should I use?”

Also ask:

“How is this engineering data structured?”

That question separates beginner database users from professional developers and data engineers.

Think Like an Engineer

Every programming problem should eventually become:

[
\boxed{\text{Problem}} \rightarrow \text{Data} \rightarrow \text{Model} \rightarrow
\text{Computation} \rightarrow \text{Decision}}
]

FAQs ?

What is [Python Programming](#) and SQL 7-in-1?

It is an integrated learning approach that combines Python programming, data structures, software development concepts, data analysis, SQL, advanced database querying, and Python-SQL integration.

Is Python or SQL better for engineering?

Neither is universally better. Python is stronger for programming, automation, numerical analysis, visualization, and machine learning, while SQL is specialized for storing, retrieving, and analysing relational data.

Should beginners learn Python before SQL?

It can be helpful, especially if your main goal is engineering programming. However, SQL can be learned independently. Eventually, combining both provides much greater capability.

Can Python connect to SQL databases?

Yes. Python applications can communicate with databases using database-specific drivers, database APIs, or higher-level tools.

Is SQL useful for mechanical engineers?

Absolutely. SQL can be useful when mechanical engineers work with manufacturing databases, sensor measurements, maintenance records, quality-control data, and production systems.

Is Python useful for civil engineers?

Yes. Python can automate calculations, analyse survey and structural data, process measurements, create visualizations, and support optimization and machine-learning workflows.

Do I need advanced mathematics to learn Python and SQL?

No. Beginners can learn the fundamentals without advanced mathematics. However, engineering applications involving simulation, statistics, optimization, or machine learning may require progressively stronger mathematical knowledge.

Can Python and SQL help with a data engineering career?

Yes. Python is useful for programming and data-processing pipelines, while SQL is fundamental for working with relational data. Together they provide an important foundation for many data-oriented roles.

Conclusion 🏆 🐍 🗄️

Python Programming and SQL 7-in-1 provides a powerful technical foundation for modern engineering because it connects programming with real-world data.

Python gives engineers the ability to **calculate, automate, analyse, simulate, visualize, and build applications**.

SQL provides the ability to **store, organize, retrieve, filter, combine, and aggregate structured information**.

The real advantage appears when both technologies are combined:

[
\boxed{\text{Python} + \text{SQL} = \text{Data-Driven Engineering}}
]

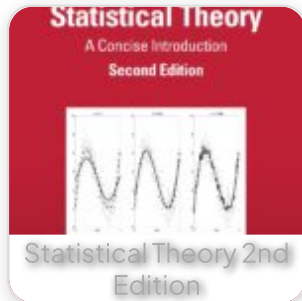
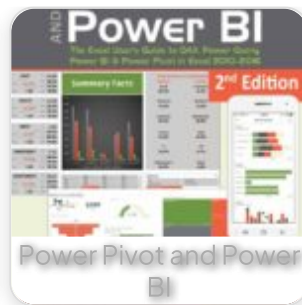
For students, this combination provides a practical path from programming fundamentals toward professional engineering applications.

For experienced engineers, it can reduce repetitive work, improve data analysis, automate reporting, and create more intelligent technical workflows.

The most effective strategy is not to study Python and SQL as isolated subjects. Instead, learn them through engineering problems—machines, sensors, structures, energy systems, manufacturing processes, measurements, and technical databases.

Learn the syntax. Understand the data. Build the system. Solve the engineering problem. 🚀 ⚙️

Related Posts:



[Preview PDF Book](#)

[← Previous Book](#)

[Next Book →](#)

EngiVerse

Explore a vast library of free engineering ebooks curated to help you expand your technical knowledge, conduct groundbreaking research, and advance your career. Our mission is to create a hub of knowledge that fosters innovation, empowers aspiring engineers, and bridges the gap between education and industry.

[Home](#) [Books](#) [About Us](#) [Contact](#)
[Privacy Policy](#) [DMCA](#)

copyright 2025 EngiVerse. All Right Reserved